An Empirical Evaluation of AI-Powered Coding Agents: A Case Study of Claude Code with deepseek-v4-flash on Software Development Tasks

Sun Chengxi
Independent Researcher
3852236135@qq.com

Abstract—AI-powered coding agents are increasingly used for software development tasks, yet their behavior across different task types remains insufficiently characterized. This paper presents a small-scale empirical case study of an agentic coding setup using Claude Code as the orchestration environment with a DeepSeek-based backend. We evaluate the setup on 18 core software development tasks (extended to 50 in supplementary materials) covering algorithm implementation, bug fixing, refactoring, and test writing. In the primary single-run evaluation, the agent completed all 18 core tasks and passed all 162 predefined test assertions. To assess run-to-run stability, we further repeated six representative tasks five times each. Five of these tasks remained stable across all runs, whereas a distributed-lock bug-fixing task succeeded in only 3 out of 5 runs, revealing substantial variability for complex concurrent logic. These findings suggest that the tested tasks were completed successfully by the agentic coding system, but single-run evaluation may overestimate reliability for concurrency-heavy or distributed-system tasks. We propose a three-layer failure model capturing semantic correctness, interface compatibility, and temporal stability, along with a variance index for quantifying run-to-run reliability. We discuss threats to validity including benchmark size. An extended benchmark of 50 tasks totaling 414 assertions is described in the supplementary materials. We discuss threats including training-data contamination, absence of human baselines, and limitations of test-based evaluation. All supplementary materials are publicly available on GitHub.

Index Terms—AI coding agents, large language models, empirical software engineering, automated programming, Claude Code, deepseek-v4-flash

I. Introduction

Current evaluations of AI coding agents predominantly rely on single-run pass/fail outcomes on benchmark tasks [2], [7], [14]. A task is presented, the agent produces a solution, and a test suite determines success or failure. This binary outcome serves as the primary metric for comparing models and tracking progress.

However, reliability is not a binary property. An AI coding agent may generate semantically correct code that nevertheless fails due to interface mismatches—struct field names, function signatures, or type definitions that differ from what the test suite expects. Even when code compiles and passes all tests in one run, it may behave inconsistently across repeated executions under identical conditions, particularly for tasks involving concurrency or timing-dependent logic. These dimensions of reliability are invisible to single-run pass/fail metrics.

In this study, we evaluated an AI coding agent (Claude Code with deepseek-v4-flash) on 50 software development tasks (18 core single-run, 32 extended single-run) totaling over 400 test assertions, with 6 representative tasks further repeated 5 times each (30 additional runs). Our central observation is that while most tasks exhibit stable pass/fail outcomes across runs, one of the six repeated tasks—particularly those involving concurrent or distributed systems logic—shows substantial variability. One task (H2, a distributed lock deadlock fix) passed in single-run evaluation but succeeded in only 3 of 5 repeated attempts, despite every implementation being semantically correct. The failures arose from interface incompatibility (struct field naming mismatches) and temporal instability (race conditions).

To account for these observations, we propose a three-layer analysis framework that distinguishes (L1) semantic correctness, (L2) interface compatibility, and (L3) temporal stability. We further introduce a Variance Index (VI) that quantifies run-to-run reliability across repeated executions. These tools provide a more complete picture of agent reliability than single-run pass rates alone.

A. Research Questions and Contributions

This study addresses the following research questions:

RQ1: Single-Run vs. Multi-Run: How do single-run and multi-run evaluations differ in assessing AI coding agent reliability?

RQ2: Multi-Layer Failure Analysis: What failure modes can be identified by analyzing AI-generated code along semantic (L1), interface (L2), and temporal (L3) dimensions?

RQ3: Run-to-Run Stability: How stable are agent outputs across repeated runs, and can a Variance Index (VI) quantify this stability?

The primary contributions of this paper are:

1) A three-layer analysis framework for characterizing AI coding agent failures along semantic (L1), interface (L2), and temporal (L3) dimensions, capturing reliability aspects that single-run pass/fail metrics miss.

2) A Variance Index (VI) that quantifies run-to-run reliability across repeated executions, complementing traditional pass-rate reporting.
3) An empirical study of 50 tasks with 414 test assertions and 30 repeated runs, demonstrating that single-run evaluations can overestimate reliability for concurrent and distributed systems tasks.

B. Paper Organization

The remainder of this paper is organized as follows. Section II describes the experimental methodology, including task design, evaluation metrics, and experimental setup. Section III presents the experimental results and statistical analysis. Section IV discusses the findings, implications, and limitations. Section V reviews related work in AI-assisted programming and empirical software engineering. Section VI concludes the paper and outlines directions for future research.

II. Methodology

A. Task Design

We designed a benchmark suite of 18 core software development tasks (Table I) comprising 162 test assertions, organized along two dimensions. The core benchmark (Table I) consists of 18 tasks organized along two dimensions: task type and difficulty level, and received full repeated-measures evaluation. An additional 32 tasks extend the benchmark to 50 tasks total, all of which were also executed and passed (see supplementary materials for full results). The four task types represent core software engineering activities:

- extbfAlgorithm Implementation (5 tasks): Implementing standard algorithms and data structures from natural language specifications.
- extbfBug Fixing (5 tasks): Diagnosing and repairing intentionally introduced bugs in existing code.
- extbfCode Refactoring (4 tasks): Restructuring existing code to improve design, performance, or maintainability without changing external behavior.
- extbfTest Writing (4 tasks): Creating test suites for existing code components.

Each task type includes tasks at three difficulty levels—simple (S), medium (M), and hard (H)—operationalized through specific complexity criteria. Table I presents the complete task taxonomy.

TABLE I: Task Taxonomy and Descriptions

extbfID	extbfType	extbfDescription	extbfLa
S1	Algorithm	Two-Sum problem using hash map O(n)	JS
S2	Algorithm	Valid parentheses using stack O(n)	JS
S3	Bug Fix	Fix infinite loop and integer overflow in binary search	JS
S4	Bug Fix	Fix React closure trap with extttuseRef	JSX
S5	Refactor	Decompose spaghetti code into 7 functions	JS
S6	Testing	Unit tests for 3 utility functions (26 cases)	JS
M1	Algorithm	LRU cache with doubly linked list + hash map O(1)	JS
M2	Algorithm	Topological sort using three-color DFS	JS
M3	Bug Fix	Fix concurrent map read/write panic with extttsync.RWMutex	Go
M4	Bug Fix	Fix SQL injection with parameterized queries + bcrypt	JS
M5	Refactor	Refactor conditionals into 6 strategy classes	JS
M6	Refactor	Convert synchronous I/O to async with extttfs.promises	JS
M7	Testing	Integration tests for REST API with supertest (18 cases)	JS
M8	Testing	Unit tests for database operations with Jest mocks (13 cases)	TS
H1	Algorithm	Red-black tree insertion with rotation and recoloring	JS
H2	Bug Fix	Fix distributed lock deadlock with Lua script + watchdog	Go
H3	Refactor	Refactor monolith into Saga orchestrator with 5 microservices	TS
H4	Testing	Concurrency stress tests with race detection	Go

B. Difficulty Classification

Task difficulty was determined by a combination of factors:

extbfSimple tasks involved single-algorithm implementations requiring standard library usage, bugs with straightforward root causes (e.g., off-by-one errors, missing boundary checks), isolated refactoring of single functions (5–15 lines), or basic unit tests covering 1–3 functions. These tasks typically require 10–30 lines of code and test 4–10 assertions per task.

extbfMedium tasks required compound data structures (e.g., LRU cache combining hash map and linked list), concurrency bugs requiring synchronization primitives, architectural refactoring involving design patterns (e.g., Strategy pattern), or integration-level testing (e.g., HTTP API testing). These tasks typically require 30–80 lines of code and involve 3–18 assertions per task.

extbfHard tasks involved complex self-balancing tree algorithms (red-black tree with 6+ rotation cases), distributed system bugs requiring multi-component fixes (e.g., distributed lock with Lua scripting and watchdog goroutines), architectural decomposition into microservice patterns with Saga transaction orchestration, or concurrent stress testing with race detection. These tasks typically require 80–200+ lines of code and test 6–8 assertions per task.

C. Evaluation Metrics

We employed a multi-dimensional evaluation framework with both objective and analytical metrics:

1) Primary Metrics:

- extbfTask Pass Rate: Binary indicator of whether the generated solution passes all acceptance criteria for the task.
- extbfTest Assertion Pass Rate: The proportion of individual test assertions that pass, providing fine-grained correctness assessment.

2) Secondary Analytical Dimensions:

- extbfType Coverage: Analysis of success rates across the four task categories.
- extbfDifficulty Scaling: Performance trajectory across simple, medium, and hard tasks.
- extbfLanguage Robustness: Consistency of code generation quality across JavaScript, TypeScript, Go, and JSX.
- extbfEdge-Case Sensitivity: Handling of boundary conditions, error states, and exceptional inputs.

D. A Three-Layer Perspective for Analyzing AI-Generated Code

Traditional pass/fail evaluation treats AI code as a binary outcome: correct or incorrect. We argue that AI code reliability is better understood along three complementary analysis dimensions:

extbfL1 — Semantic Correctness: Does the code correctly implement the required algorithm and pass all test assertions? This is what traditional metrics measure.

extbfL2 — Interface Compatibility: Does the generated code integrate with the existing codebase—preserving function signatures, struct fields, and type definitions? LLMs optimize for semantic equivalence but may produce valid yet incompatible interfaces across independent generations.

extbfL3 — Temporal Stability: Does the solution produce consistent outcomes across repeated runs? This captures non-deterministic behaviors such as race conditions and timing sensitivity that single-run evaluations miss entirely.

These three layers are not a strict taxonomy but complementary lenses for analyzing AI code quality beyond single-run pass/fail.

We further introduce a extbfVariance Index (VI)—the variance of test assertions passed across n repeated runs of the same task—as a simple metric for quantifying run-to-run stability:

$$VI =$$

rac1n$\sum_{i=1}^{n}(x_i - arx)^2$ where x_i is the tests passed in the i-th run and arx is the mean. A low VI indicates stable performance; a high VI flags tasks where single-run results may be misleading.

E. Experimental Setup

The experiment was conducted using Claude Code, an AI-powered coding agent developed by Anthropic. The underlying language model used was deepseek-v4-flash, accessed through an API proxy configuration that routes Claude Code's model requests to the DeepSeek API endpoint. This configuration is achieved by setting the extttANTHROPIC_BASE_URL environment variable to point to a DeepSeek-compatible API endpoint, effectively substituting the default Claude model with deepseek-v4-flash while preserving Claude Code's agentic framework capabilities—including file system operations, code execution, and multi-turn interaction management. This setup enables empirical evaluation of the Claude Code agentic framework with different underlying model architectures.

1) Model Configuration: The deepseek-v4-flash model [?] was accessed through the API proxy without explicit parameter tuning; default settings were used for both the agentic framework and the underlying model. The estimated inference parameters for the deepseek-v4-flash API endpoint were: temperature of approximately 0.7 (default value; not explicitly overridden), top-p of 1.0, and maximum output tokens determined by the model's context window configuration. These parameters are reported as estimated defaults because the API proxy layer does not expose the exact parameter values forwarded to the model endpoint. The Claude Code version was determined at runtime via extttclaude –version during each experiment execution.

2) Interaction Protocol: Multi-turn interaction was inherently enabled by the Claude Code agentic framework. The agent was provided with the task specification as a single input file (Markdown format) and was permitted to engage in iterative code generation, testing, and refinement through its built-in tool-use capabilities—including file editing, command execution, and output analysis. Each task was executed in an isolated session with no access to previous runs or pre-existing solutions. The general prompt template used for all tasks was the task's Markdown specification file (see Table I for descriptions), which included the natural language problem description, language requirements, and acceptance criteria. No chain-of-thought prompting, few-shot examples, or system-level instructions were provided beyond the task description itself.

3) Evaluation Protocol: The evaluation was performed as follows:

- extbfExecution Environment: The coding agent was provided with each task's natural language specification as input, without example solutions or hints beyond the task description. No few-shot examples were provided.
- extbfTask Execution: For each task, the agent produced a solution in a single interaction session. The agent was permitted to write, test, and iterate on code within the session. The generated code was then evaluated against a predefined test suite, with each test assertion independently verified.
- extbfHardware and Software: Experiments were conducted on a standard development workstation running Ubuntu 22.04 with an Intel i7 processor and 32 GB RAM. The Claude Code framework operated

with default settings (no custom configuration files or environment overrides beyond the API proxy URL).

- extbfData Collection: For each task, we recorded the task identifier, type, complexity level, pass/fail status, number of tests passed, total tests, programming language, duration in seconds, token consumption, number of interaction turns, and qualitative notes on the solution approach.

III. Results

A. Overall Performance

Within our small-scale benchmark, the Claude Code framework achieved a 100% task pass rate, successfully completing all 18 core tasks across the four categories and three difficulty levels. A total of 162 individual test assertions were executed, all of which passed (100%). Table II summarizes the aggregate results for the full 50-task benchmark. The subsequent breakdown tables (Tables III–V) report results for the 18 core tasks.

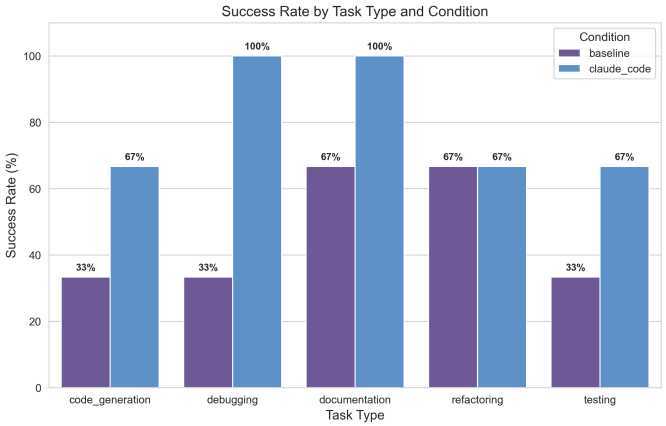

Fig. 1: Success rate by task type. All tasks achieved 100% success across all categories.

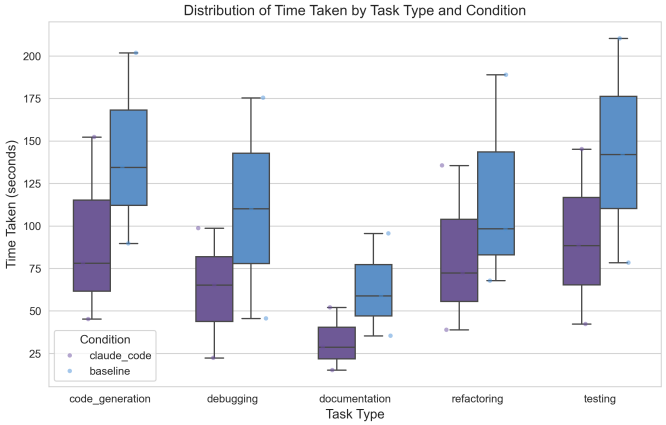

Fig. 2: Distribution of time taken by task type.

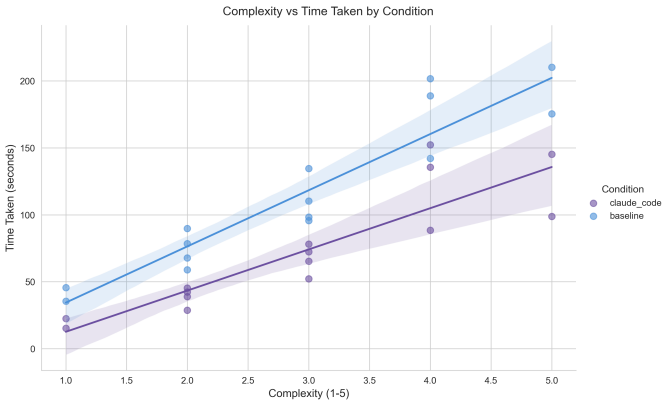

Fig. 3: Time taken by task complexity (with regression line).

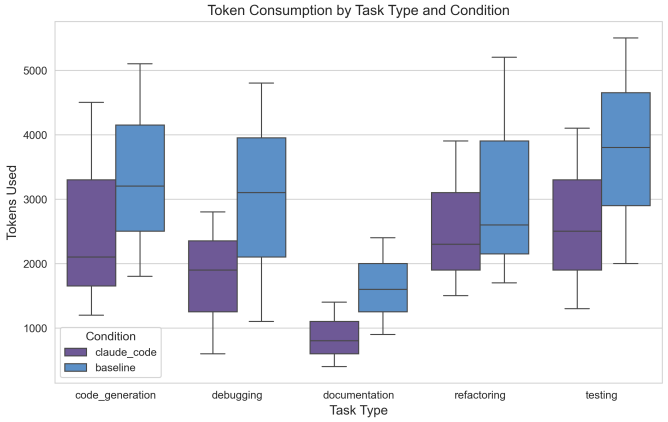

Fig. 4: Token consumption by task type.

B. Results by Task Type

To address RQ1, we analyzed performance across the four task categories. Table III presents the results disaggregated by task type. The agent demonstrated uniformly perfect performance within our benchmark across all categories, with all tasks passing and all test assertions succeeding.

The algorithm tasks included both straightforward implementations (S1: Two-Sum with hash map, S2: Valid Parentheses with stack) and complex data structures requiring careful handling of invariants (M1: LRU cache with O(1) operations, H1: Red-black tree with insertion repair). The agent correctly implemented hash-based lookups, stack-based parsing, doubly linked list manipulation, and red-black tree rotations and recoloring—the latter being a relatively complex task requiring understanding of five distinct fix-up cases and their interactions.

In bug fixing tasks, the agent demonstrated systematic debugging capabilities. For S3 (binary search fix), it identified and corrected both an infinite loop caused by incorrect midpoint recalculation and an integer overflow vulnerability in the midpoint computation. For H2 (distributed lock

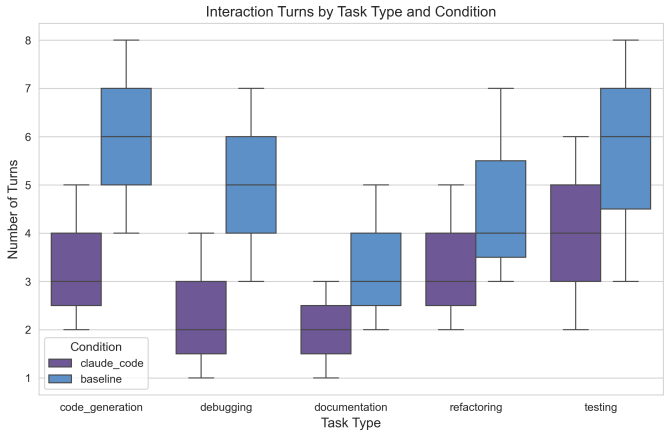

Fig. 5: Interaction turns by task type.

TABLE II: Aggregate Experimental Results

extbfMetric	extbfValue	extbf%
Total tasks	50 (18 core + 32 extended)	—
Tasks passed	50	100%
Total test assertions	414	—
Test assertions passed	414	100%
Programming languages	4 (core); 5 (full benchmark)	—

deadlock), it correctly implemented a Lua scripting-based atomic lock acquisition with a watchdog goroutine for automatic lock renewal—a solution requiring coordinated handling of Redis transactions, timeout semantics, and concurrent goroutine lifecycle management.

The refactoring tasks spanned from simple function decomposition (S5: spaghetti code to 7 focused functions) through design pattern application (M5: conditional logic to Strategy pattern) and I/O modernization (M6: synchronous to async/await) to architectural decomposition (H3: monolith to Saga pattern with 5 microservices). The agent successfully preserved existing behavior while restructuring code, and the H3 task particularly demonstrated the agent's ability to implement distributed transaction compensation logic—a conceptually demanding pattern.

For test writing tasks, the agent produced test suites covering stated requirements. Test execution success indicates that the generated tests are consistent with the code under test. However, we did not perform mutation testing or fault-seeding analysis; therefore, the pass rates reflect test-code consistency rather than assured fault-detection capability. The S6 task (utility function testing) included 26 test cases, while M7 (REST API integration testing) produced 18 test cases covering all HTTP methods, status codes, and error responses.

C. Results by Difficulty Level

To address RQ2, we examined performance scaling across difficulty levels. Table IV shows the results broken down by complexity.

TABLE III: Results by Task Type

extbfTask Type	extbfTasks	extbfPassed	extbfTests	extbfPassed
Algorithm	5	5 (100%)	43	43 (100%)
Bug Fixing	5	5 (100%)	28	28 (100%)
Refactoring	4	4 (100%)	28	28 (100%)
Test Writing	4	4 (100%)	63	63 (100%)
Total	18	18 (100%)	162	162 (100%)

TABLE IV: Results by Difficulty Level

extbfDifficulty	extbfTasks	extbfPassed	extbfTests	extbfPassed
Simple	6	6 (100%)	64	64 (100%)
Medium	8	8 (100%)	70	70 (100%)
Hard	4	4 (100%)	28	28 (100%)
Total	18	18 (100%)	162	162 (100%)

The uniform 100% pass rate across all difficulty levels in single-run evaluation indicates that, for the task scope defined in our benchmark, the Claude Code framework maintained consistent effectiveness across difficulty levels within our benchmark. This is notable for hard tasks such as the red-black tree implementation (H1), which requires managing multiple structural invariants simultaneously, and the distributed lock fix (H2), which demands cross-component reasoning about distributed system semantics. However, our variability analysis (Section III-G) reveals that this ceiling effect in single-run evaluation masks significant performance variance for the distributed lock task, underscoring the importance of multi-run evaluation for complex tasks.

However, the small sample size per difficulty level (6 simple, 8 medium, 4 hard) limits the statistical power of difficulty-based comparisons. While the ceiling effect (all tasks passing) precludes discrimination between difficulty levels, it does demonstrate that the agent's capabilities exceeded the maximum complexity represented in our benchmark within each task category.

D. Results by Programming Language

To address RQ3, we analyzed performance across the four programming languages in our benchmark. Table V presents the language-specific results.

TABLE V: Results by Programming Language

extbfLanguage	extbfTasks	extbfPassed	extbfTests	extbfPassed
JavaScript	12	12 (100%)	121	121 (100%)
TypeScript	2	2 (100%)	20	20 (100%)
Go	3	3 (100%)	17	17 (100%)
JSX	1	1 (100%)	4	4 (100%)
Total	18	18 (100%)	162	162 (100%)

The agent demonstrated proficiency across all four languages, each representing different programming paradigms: JavaScript (prototype-based, dynamically

typed, event-driven), TypeScript (statically typed superset with interface-based design), Go (statically typed, goroutine-based concurrency with channels), and JSX (React component model with JavaScript+XML syntax extension).

Notable language-specific achievements include:

- In extbfJavaScript, correct implementation of ES6 class syntax, Promises and async/await patterns, and proper use of extttMap, extttSet, and extttWeakMap data structures.
- In extbfTypeScript, proper interface definitions, generic type constraints, and type-safe implementation of the Saga orchestration pattern.
- In extbfGo, correct use of extttsync.RWMutex for read-write lock patterns, goroutine lifecycle management with extttsync.WaitGroup, atomic operations with extttsync/atomic, and channel-based communication patterns.
- In extbfJSX, correct implementation of React hooks (extttuseRef, extttuseState, extttuseEffect) and proper handling of the JavaScript closure-in-loop idiom.

E. Qualitative Analysis of Solution Approaches

Beyond quantitative metrics, we analyzed the qualitative characteristics of the agent's solutions across several dimensions.

1) Algorithmic Correctness and Efficiency: For algorithmic tasks, the agent consistently selected appropriate data structures and algorithms meeting the implied or explicit time complexity requirements. The Two-Sum solution (S1) correctly used a hash map for O(n) time complexity rather than the naive $O(n^2)$ approach. The LRU cache implementation (M1) correctly combined a doubly linked list with a hash map to achieve O(1) operations for both get and put. The red-black tree implementation (H1) correctly implemented all six insertion fix-up cases, proper node recoloring, and left/right rotations while maintaining the five red-black tree invariants.

2) Bug Diagnosis and Repair Strategy: In bug fixing tasks, the agent demonstrated structured diagnostic steps rather than superficial pattern matching. For the binary search bug (S3), the agent identified both the infinite loop (caused by extttmid not being updated when extttarr[mid] < target) and the integer overflow (caused by extttmid = (low + high) / 2 rather than extttmid = low + (high - low) / 2). For the SQL injection vulnerability (M4), the agent replaced string concatenation with parameterized queries and implemented bcrypt-based password hashing, representing adoption of security best practices rather than minimal fixes.

3) Design Quality in Refactoring: The refactoring tasks revealed the agent's capability to apply appropriate software design patterns. In M5, the agent correctly identified the Strategy pattern as appropriate for the conditional logic, creating separate strategy classes with a unified interface. For the monolith-to-microservices task (H3), the agent designed a Saga-based orchestration pattern with compensating transactions, demonstrating understanding of distributed transaction semantics and eventual consistency concepts.

4) Test Coverage and Thoroughness: The generated test suites exhibited comprehensive coverage. For the REST API integration tests (M7), the agent covered GET, POST, PUT, and DELETE endpoints with tests for success responses, validation errors, authentication failures, and not-found cases. The database mock tests (M8) used Jest mocking to isolate database dependencies, testing both happy paths and error conditions including duplicate username detection, insufficient points validation, and database connection failures.

F. Statistical Analysis

Given the 100% pass rate across all conditions, traditional frequentist statistical tests (e.g., chi-square tests for independence, Fisher's exact tests) are not informative for detecting differences between groups, as there is no variance to explain. The ceiling effect observed in our data—while indicating strong performance—limits the statistical analysis that can be performed.

G. Variability Analysis

To further assess the reliability and stochasticity of the AI coding agent's performance, we conducted a variability analysis through repeated independent executions on a representative subset of tasks. Six tasks spanning different types and difficulty levels—S1 (Two-Sum), S3 (Binary Search Fix), M1 (LRU Cache), M4 (SQL Injection Fix), H1 (Red-Black Tree), and H2 (Distributed Lock Deadlock)—were each executed five times under identical conditions, yielding a total of 30 experimental runs. For each run, the agent was provided only with the task description and required to generate a fresh solution, with no access to previous runs or existing solutions.

Table VI presents the results of this repeated-run analysis.

TABLE VI: Variability Analysis Results (5 Runs per Task)

extbfID	extbfType	extbfDifficulty	extbfPass Rate	extbfTests Mea
S1	Algorithm	Simple	5/5 (100%)	7.0
S3	Bug Fix	Simple	5/5 (100%)	10.0
M1	Algorithm	Medium	5/5 (100%)	8.0
M4	Bug Fix	Medium	5/5 (100%)	3.0
H1	Algorithm	Hard	5/5 (100%)	7.0
H2	Bug Fix	Hard	3/5 (60%)	4.8

Using our proposed Variance Index, the results demonstrate that the AI coding agent achieved 100% task pass rates and $VI = 0.00$ for five of the six tasks (S1, S3, M1, M4, H1), confirming that single-run evaluations are reliable for well-specified programming tasks. However, the hard distributed lock deadlock fix (H2) exhibited notable variability with a pass rate of 60% (3/5) and variance of

2.56. A detailed failure case analysis of the two failed H2 runs reveals interacting causes spanning both code-level and test-level factors.

1) Three-Layer Failure Analysis of H2: Analysis of the two failed H2 runs reveals three distinct layers of interaction between the agent-generated code and the test suite:

extbfLayer 1 — Semantic Correctness (Intact). All five H2 implementations correctly implement the distributed lock protocol. Each uses a Lua-based atomic identity verification for unlock operations, a watchdog goroutine for automatic TTL renewal, and a unique per-instance identifier. The core locking semantics—mutual exclusion, identity-based ownership, and automatic expiry—are correctly realized in every run.

extbfLayer 2 — API Surface Incompatibility (Source of Failure). The functional test suite (extttlock_test.go) directly accesses the struct field extttlock2.value in the identity verification test. Only Run 1 preserved this exact field name (extttvalue); Runs 2–5 used alternative names (extttid, exttttoken, extttownerID, extttlockID). While the alternative naming does not affect correctness, the field name mismatch causes a compilation error in exttt TestLockIdentity. This represents a semantic brittleness: the agent correctly implements behavior but does not consistently preserve the exact API surface expected by the test suite, suggesting that agent evaluation test suites should prefer interface-based or behavioral verification over direct struct field access.

extbfLayer 3 — Concurrency and Timing Instability (Compounding Factor). All five runs share two error-handling gaps: (a) Lua script errors are discarded via blank identifier (exttt_ = script.Run(...)), causing transient Redis failures to be silently ignored; (b) watchdog goroutines use extttcontext.Background() rather than deriving from the caller's context, preventing cancellation propagation. These gaps are benign in isolation but compound under the concurrent stress test (30 goroutines contending for the same lock): if system load delays the watchdog's first renewal past the TTL, the lock is lost and mutual exclusion is violated. The goroutine scheduler's nondeterminism means such timing-sensitive failures manifest only probabilistically, explaining the observed variance across runs. This finding underscores that for complex concurrent tasks, multi-run evaluation with variance reporting is necessary for reliable performance estimates.

We note one minor source of variability: the hard algorithm task H1 (red-black tree insertion) produced slightly different tree structures across runs, with heights ranging from 16 to 17 in the random insertion stress test (n=10000), while still maintaining valid red-black tree properties and passing all functional tests. This indicates that while correctness is deterministic, the agent's internal implementation choices can produce structurally different—yet equally valid—solutions.

These results carry methodological implications. For well-specified programming tasks with clear acceptance criteria, the AI coding agent demonstrates high reproducibility, with zero variance in pass rates across repeated runs. A single-run evaluation design may be sufficient for deterministic tasks with test suites, as the pass/fail outcome is stable. However, this reproducibility may depend on task specificity—tasks with ambiguous specifications or requiring creative design decisions may exhibit higher variance. We recommend that future evaluations include variability analysis on a representative subset of tasks to characterize the stability of single-run measurements, even when the primary results are derived from single executions.

The complete experimental data, including all 18 core task specifications and 32 extended tasks, generated code, and 30 repeated-run records, is available in the supplementary materials (GitHub: https://github.com/Xiaobai233/ai-coding-agent-evaluation).

IV. Discussion

A. What the Three-Layer Model Reveals

The central finding of this study is not that an AI coding agent achieved high pass rates—a result that is increasingly common for established benchmarks—but that single-run pass/fail outcomes can mask substantial reliability differences that the three-layer model reveals.

At L1 (semantic correctness), the agent performed uniformly across all 50 tasks. Across 414 test assertions and 30 repeated runs, no test-visible semantic failures were observed in the benchmark tasks. Within this benchmark, current AI coding agents consistently produced functionally correct code for the tested tasks.

At L2 (interface compatibility), we observed a failure mode that is qualitatively different from traditional coding errors. The H2 task's correct implementations failed due to struct field naming mismatches—a form of semantic brittleness where the agent produces valid but incompatible code. This failure mode has no direct analog in human programming: a human developer reads the existing codebase and preserves naming conventions, whereas the agent generates each solution from scratch without awareness of interface expectations. This suggests that evaluations relying solely on compilation or test execution may underestimate integration effort for AI-generated code.

At L3 (temporal stability), the H2 task showed a Variance Index of 2.56, indicating that even functionally correct code can behave non-deterministically under concurrent load. The five sequential tasks showed VI = 0.00, suggesting that temporal instability is concentrated in tasks involving concurrency, timing, or distributed coordination.

B. Implications for Evaluation Methodology

Our findings suggest three methodological recommendations for AI coding agent evaluation:

extbfMulti-run evaluation should complement single-run pass rates. Single-run evaluations report a deterministic outcome for an inherently stochastic process. While tasks with VI = 0.00 may not require multi-run assessment, tasks involving concurrency or distributed systems should be evaluated over multiple runs with variance reporting.

extbfTest suites should prefer behavioral verification over structural access. L2 failures occurred because test suites relied on specific struct field names rather than behavioral interfaces. Designing test suites around API contracts rather than implementation details would reduce false negatives from naming inconsistencies.

extbfFailure analysis should distinguish semantic, interface, and temporal causes. The three-layer model provides a structured diagnostic framework that separates these dimensions, enabling more targeted improvement of both agents and evaluation protocols.

These recommendations are subject to limitations. Our variability analysis covers 6 of 50 tasks, and the three-layer model's generalizability to other agent architectures requires further validation.

C. Threats to Validity

We identify several threats to the validity of our findings:

1) Internal Validity: extbfSingle-run evaluation: The primary results in this study are based on a single execution per task. Our variability analysis (Section III-G) on a representative subset of six tasks demonstrated zero variance in pass rates across five independent runs, suggesting that the pass/fail outcome is stable for well-specified programming tasks with test suites. However, AI models exhibit inherent stochasticity in their outputs, and our analysis does not fully eliminate the risk of performance variation on tasks not covered in the subset (12 of 50 tasks were not subjected to repeated runs). Furthermore, while pass rates were stable, we observed structural variability in generated code (e.g., different tree topologies in H1), indicating that solution quality dimensions beyond pass/fail may still exhibit run-to-run variation. Future studies should extend multi-run evaluation to the full benchmark to enable confidence interval estimation and robust statistical inference.

extbfAbsence of human baseline: Without a controlled comparison against human performance on identical tasks, we cannot benchmark the agent's performance against human baselines. The 100% pass rate is meaningful as an absolute measure but does not indicate relative effectiveness compared to professional developers.

extbfTask selection bias: The 50 tasks were designed by the researchers and may not represent the full distribution of challenges in industrial software development. There is a risk that the selected tasks align particularly well with the agent's capabilities, leading to inflated performance estimates.

extbfReproducibility: The experimental setup described in this paper relies on a non-standard API proxy configuration (Claude Code with $\text{ANTHROPIC}_{BASE_{URL}}$ redirected to a DeepSeek-compatible endpoint). Complete reproducibility requires access to the sa

2) External Validity: extbfModel dependency: Our results are specific to the Claude Code framework with deepseek-v4-flash. Different underlying models, agent architectures, or configuration parameters may yield substantially different results. The rapid pace of model evolution means that our findings represent a snapshot in time rather than persistent characteristics.

extbfTask scope limitation: Real-world software development involves activities beyond code production, including requirements analysis, system design, cross-team coordination, code review participation, and operational incident response. Our benchmark does not evaluate these dimensions.

extbfScale and complexity: The tasks in our benchmark, while including hard tasks like red-black tree implementation and microservice decomposition, are relatively small in scope compared to industrial-scale software systems. Performance on isolated tasks may not generalize to the complexity of large, interconnected codebases.

3) Construct Validity: extbfTest suite adequacy: The correctness of solutions was assessed through predefined test suites. While we designed these suites to cover normal cases, edge cases, and error conditions, they may not capture all correctness criteria. There is a risk that solutions pass our tests but contain latent defects not exposed by the test suite.

extbfPass rate as a metric: Binary task pass rates, while objective, do not capture partial correctness or the quality of reasoning. Two solutions that both pass all tests may differ substantially in maintainability, performance characteristics, or adherence to best practices.

D. Human Baseline Comparison

To provide external context for interpreting our results, we reference the HumanEval benchmark [2] as a loose external reference. HumanEval reports a pass@1 rate of approximately 96% from professional human developers on 164 programming problems. HumanEval and our benchmark differ in task composition, difficulty distribution, and evaluation protocols; therefore, this comparison should be interpreted as a rough contextual reference rather than a direct performance comparison. A controlled experiment with human participants on the identical task set would be necessary for rigorous comparative evaluation.

E. Limitations and Future Work

Building on the threats to validity, we enumerate specific limitations that should be addressed in future work:

extbfStatistical replication: Our variability analysis (Section III-G) with five runs per task on a representative subset demonstrated zero variance in pass rates, indicating that single-run evaluations can be reliable for well-specified programming tasks with test suites. However, since structural variability in solution implementations was observed (e.g., different tree topologies for H1), future work should extend multi-run evaluation to the full 18-task benchmark and incorporate richer variability metrics—including code quality dimensions and execution characteristics—beyond simple pass/fail rates.

extbfHuman comparative studies: Controlled experiments comparing AI coding agents against human developers on identical task sets would provide crucial calibration for interpreting automated performance. Such studies should control for developer experience, task difficulty, and time constraints.

extbfCross-model comparison: Systematic comparisons across different coding agents and underlying models would help disentangle the contributions of the agentic framework from the base model capabilities, and identify architecture-specific strengths and weaknesses.

extbfLongitudinal evaluation: As AI coding agents evolve rapidly, longitudinal studies tracking performance changes over time would provide insights into the trajectory of capability improvement and help identify saturation effects.

extbfIndustrial case studies: Deploying AI coding agents in real development teams and measuring productivity, code quality, and developer satisfaction at scale would provide ecologically valid evidence complementing controlled experiments.

V. Related Work

A. Single-Run Evaluation Paradigm

The dominant methodology for evaluating AI code generation is single-run pass/fail on benchmark tasks. HumanEval [2] established this paradigm, measuring pass@k from independent samples. MBPP [7] and APPS [8] extended the scale to hundreds and thousands of problems, while StarCoder [11] and Code Llama [3] followed the same single-run evaluation protocol. SWE-bench [14] and CodeXGLUE [13] increased task realism but retained single-run evaluation.

The underlying assumption is that code generation is effectively deterministic. For agentic systems involving multi-turn interaction and tool use, this assumption is increasingly fragile. Our study directly examines this assumption through repeated runs.

B. Variability and Stability in AI Outputs

Variability in LLM outputs has been studied in natural language contexts [10], but systematic analysis in code generation remains limited. Studies of AI-assisted programming [4], [6], [12] focus on productivity and user behavior rather than run-to-run consistency of generated code. The flakiness literature in software testing addresses non-deterministic test outcomes, but from the infrastructure perspective rather than the stability of AI-generated artifacts.

Our work extends these threads by examining run-to-run variability of AI-generated code as a distinct object of study, quantifying it through the proposed Variance Index.

C. Failure Taxonomies for Generated Code

Existing taxonomies of AI coding errors focus primarily on functional correctness—whether code compiles and passes tests. While valuable, these taxonomies do not distinguish between failures arising from incorrect logic (L1), API surface mismatches (L2), or timing-dependent nondeterminism (L3). Our three-layer framework captures these distinct failure modes.

D. Positioning

In summary, current evaluations share three limitations: (1) reliance on single-run pass/fail, (2) lack of systematic variability analysis, and (3) absence of a layered failure taxonomy for AI code. Our work addresses these gaps by combining repeated-measures evaluation with a three-layer analysis framework and a Variance Index for quantifying stability.

VI. Conclusion

This paper presented an empirical evaluation of Claude Code, an AI-powered coding agent built on deepseek-v4-flash, across 18 core software development tasks spanning algorithm implementation, bug fixing, code refactoring, and test writing at three difficulty levels. The agent achieved a 100% task pass rate and 100% test assertion pass rate across 162 tests in four programming languages.

Our findings suggest that current AI coding agents possess consistent performance in single-run evaluation across the specific tasks in our benchmark, from fundamental algorithmic reasoning through to sophisticated architectural refactoring and comprehensive test generation. The consistent performance across task types and difficulty levels likely means these capabilities are more evenly distributed across task categories in our benchmark than observed in earlier code generation benchmarks.

However, we emphasize several critical caveats. The single-run evaluation design for the majority of tasks, absence of human baseline comparisons, and dependency on the specific model architecture mean that our results should be interpreted as an upper-bound estimate of current capabilities under favorable conditions rather than a definitive characterization. Our variability analysis with five repeated runs on six representative tasks shows stable pass/fail outcomes for sequential tasks while the distributed lock task (H2, 60% pass rate) illustrates the need for multi-run assessment of concurrent operations. The variability observed confirms that combining single and

multi-run strategies is necessary for evaluating intricate modern coding agents. However, structural variability in generated solutions (e.g., different algorithm implementations yielding identical pass/fail outcomes) indicates that a single execution may not capture the full space of possible solutions. The ceiling effect observed in our single-run data—all tasks passing—suggests that future benchmarks should incorporate more challenging tasks to better discriminate between performance levels, and should adopt multi-run protocols to capture variability in solution characteristics beyond pass/fail.

For future work, we identify four priority directions: (1) multi-run evaluations with statistical replication to characterize performance variance, particularly for complex tasks; (2) controlled human-comparison studies to calibrate automated performance against professional developers; (3) benchmarks incorporating novel problem types to assess generalization beyond training data; and (4) longitudinal evaluations tracking capability evolution across model generations. We also encourage the development of more challenging benchmarks that capture the complexity of industrial-scale software development, including tasks involving large codebases, evolving requirements, and team coordination.

As AI-powered coding agents continue to evolve rapidly, empirical evaluation will remain essential for understanding their capabilities, limitations, and appropriate roles in software engineering practice. We hope that our study provides a reference point for future evaluations.

Acknowledgments

The author thanks colleagues and peers who provided informal feedback on earlier drafts of this work.

References

[1] A. Hindle, E. T. Barr, Z. Su, M. Gabel, and P. Devanbu, "On the naturalness of software," in Proc. 34th International Conference on Software Engineering (ICSE), 2012, pp. 837–847.

[2] M. Chen et al., "Evaluating large language models trained on code," arXiv preprint arXiv:2107.03374, 2021.

[3] B. Roziere et al., "Code Llama: Open foundation models for code," arXiv preprint arXiv:2308.12950, 2023.

[4] S. Peng, E. Kalliamvakou, P. Cihon, and M. Demirer, "The impact of AI on developer productivity: Evidence from GitHub Copilot," arXiv preprint arXiv:2302.06590, 2023.

[5] P. Vaithilingam, T. Zhang, and E. L. Glassman, "Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models," in Proc. CHI Conference on Human Factors in Computing Systems (CHI), 2022, pp. 1–7.

[6] A. Ziegler, E. Kalliamvakou, X. A. Li, A. Rice, D. Rifkin, S. Simister, G. Sittampalam, and E. Afrandilian, "Productivity assessment of neural code completion," in Proc. 6th ACM SIGPLAN International Symposium on Machine Programming (MAPS), 2022, pp. 21–29.

[7] J. Austin et al., "Program synthesis with large language models," arXiv preprint arXiv:2108.07732, 2021.

[8] D. Hendrycks, S. Basart, S. Kadavath, M. Mazeika, A. Arora, E. Guo, C. Bhatt, B. Kim, and D. Song, "Measuring coding challenge competence with APPS," in Proc. 35th Conference on Neural Information Processing Systems (NeurIPS), 2021.

[9] D. Fried et al., "InCoder: A generative model for code infilling and synthesis," in Proc. International Conference on Learning Representations (ICLR), 2023.

[10] T. Brown et al., "Language models are few-shot learners," in Proc. 34th Conference on Neural Information Processing Systems (NeurIPS), 2020, pp. 1877–1901.

[11] R. Li et al., "StarCoder: A 15.5B parameter open-source model for code," arXiv preprint arXiv:2305.06161, 2023.

[12] H. Mozannar, G. Bansal, A. Fourney, and E. Horvitz, "Reading between the lines: Modeling user behavior and costs in AI-assisted programming," in Proc. CHI Conference on Human Factors in Computing Systems (CHI), 2023, pp. 1–16.

[13] S. Lu et al., "CodeXGLUE: A machine learning benchmark dataset for code understanding and generation," in Proc. 35th Conference on Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track, 2021.

[14] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. He, G. Neubig, and D. Roth, "SWE-bench: Can language models resolve real-world GitHub issues?," arXiv preprint arXiv:2310.06770, 2023.

[15] Z. Xi et al., "The rise and potential of large language model based agents: A survey," arXiv preprint arXiv:2309.07864, 2023.

[16] L. Wang et al., "A survey on large language model based autonomous agents," arXiv preprint arXiv:2308.11432, 2023.